

Entrega Actividad 6: Ejercicios Resueltos y Propuestos

1. Ejercicio 2.10. Sobrecarga de métodos

1.1. Enunciado (Ejemplo del Texto): Clase Pedido

El ejemplo muestra una clase Pedido con un método sobrecargado calcularPedido que recibe diferente cantidad de parámetros para calcular el total de los platos.

1.1.1. Código Fuente

```
1  /**
2   * Esta clase define objetos de tipo Pedido de un restaurante que consta
3   * de diferentes platos y que tiene un determinado valor.
4   * @version 1.2/2020
5   */
6  public class Pedido {
7      /**
8       * Método que tiene como parámetros los elementos que conforman
9       * un pedido con un primer plato, su bebida y sus costos
10      * correspondientes. El método calcula el costo total del pedido y
11      * muestra en pantalla los datos del pedido y su costo total.
12      * @param primerPlato Parámetro que define el nombre del primer
13      * plato que conforma el pedido
14      * @param costoPrimerPlato Parámetro que define el costo del
15      * primer plato que conforma el pedido
16      * @param bebida Parámetro que define el nombre de la bebida que
17      * conforma el pedido
18      * @param costoBebida Parámetro que define el costo de la bebida
19      * que conforma el pedido
20      */
21      public void calcularPedido(String primerPlato, double costoPrimerPlato, String bebida, double
22          ↪ costoBebida) {
23          double total = costoPrimerPlato + costoBebida;
24          System.out.println("El costo de " + primerPlato + " y " + bebida + " es = $" + total);
25      }
26
27      /**
28       * Método sobrecargado que tiene como parámetros los elementos
29       * que conforman un pedido con un primer plato, un segundo plato,
30       * su bebida y sus costos correspondientes. El método calcula el costo
31       * total del pedido y muestra en pantalla los datos del pedido y su
32       * costo total.
33       * @param primerPlato Parámetro que define el nombre del primer
34       * plato que conforma el pedido
35       * @param costoPrimerPlato Parámetro que define el costo del
36       * primer plato que conforma el pedido
37       * @param segundoPlato Parámetro que define el nombre del
38       * segundo plato que conforma el pedido
39       * @param costoSegundoPlato Parámetro que define el costo del
40       * segundo plato que conforma el pedido
41       * @param bebida Parámetro que define el nombre de la bebida que
42       * conforma el pedido
43       * @param costoBebida Parámetro que define el costo de la bebida
44       * que conforma el pedido
45      */
46      public void calcularPedido(String primerPlato, double costoPrimerPlato, String segundoPlato, double
47          ↪ costoSegundoPlato, String bebida, double costoBebida) {
48          double total = costoPrimerPlato + costoSegundoPlato + costoBebida;
49          System.out.println("El costo de " + primerPlato + " + " + segundoPlato + " + " + bebida + " es = $"
50          ↪ " + total);
51      }
52
53      /**
54       * Método sobrecargado que tiene como parámetros los elementos
55       * que conforman un pedido con un primer plato, segundo plato, su
56       * bebida, postre y sus costos correspondientes. El método calcula el
57       * costo total del pedido y muestra en pantalla los datos del pedido y
58       * su costo total.
59       * @param primerPlato Parámetro que define el nombre del primer
60       * plato que conforma el pedido
61       * @param costoPrimerPlato Parámetro que define el costo del
```

```

59      * primer plato que conforma el pedido
60      * @param segundoPlato Parámetro que define el nombre del
61      * segundo plato que conforma el pedido
62      * @param costoSegundoPlato Parámetro que define el costo del
63      * segundo plato que conforma el pedido
64      * @param postre Parámetro que define el nombre del postre que
65      * conforma el pedido
66      * @param costoPostre Parámetro que define el costo del postre que
67      * conforma el pedido
68      * @param bebida Parámetro que define el nombre de la bebida que
69      * conforma el pedido
70      * @param costoBebida Parámetro que define el costo de la bebida
71      * que conforma el pedido
72      */
73      public void calcularPedido(String primerPlato, double costoPrimerPlato, String segundoPlato, double
74          ↳ costoSegundoPlato, String postre, double costoPostre, String bebida, double costoBebida) {
75          double total = costoPrimerPlato + costoSegundoPlato + costoBebida + costoPostre;
76          System.out.println("El costo de " + primerPlato + " + " + segundoPlato + " + " + bebida + " + " +
77          ↳ postre + " es = $" + total);
78      }
79      /**
80      * Método main que crea tres diferentes tipos de pedidos en el
81      * restaurante.
82      */
83      public static void main (String args[]) {
84          Pedido pedido1 = new Pedido();
85          pedido1.calcularPedido("Sancocho", 5000, "Gaseosa", 2000);
86          Pedido pedido2 = new Pedido();
87          pedido2.calcularPedido("Crema de verduras", 5000, "Churrasco", 6000, "Gaseosa", 2000);
88          Pedido pedido3 = new Pedido();
89          pedido3.calcularPedido("Crema de espinacas", 5000, "Salmón", 10000, "Tiramisú", 5000, "Gaseosa",
90          ↳ 2000);
91      }
92  }

```

1.1.2. Diagrama de clases

Pedido
+ calcularPedido(primerPlato: String, costoPrimerPlato: double, bebida: String, costoBebida: double): void + calcularPedido(primerPlato: String, costoPrimerPlato: double, segundoPlato: String, costoSegundoPlato: double, bebida: String, costoBebida: double): void + calcularPedido(primerPlato: String, costoPrimerPlato: double, segundoPlato: String, costoSegundoPlato: double, postre: String, costoPostre: double, bebida: String, costoBebida: double): void + main(args: String[])\$

1.1.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_2_10/Pedido.java

1.2. Ejercicio Propuesto: Clase Suma

Desarrollo de una clase denominada Suma, la cual posee múltiples métodos sumar con el mismo nombre pero con diferentes cantidades o tipos de parámetros (enteros y dobles).

1.2.1. Código Fuente

```

1  package Ejercicio_2_10;
2
3  public class Suma {
4      public int sumar(int a, int b) {
5          return a + b;
6      }
7
8      public int sumar(int a, int b, int c) {
9          return a + b + c;
10     }
11
12     public double sumar(double a, double b) {
13         return a + b;
14     }
15
16     public double sumar(double a, double b, double c) {
17         return a + b + c;
18     }
19
20     public static void main(String[] args) {
21         Suma suma = new Suma();
22         System.out.println("Suma 2 ints (2+3): " + suma.sumar(2, 3));
23         System.out.println("Suma 3 ints (2+3+4): " + suma.sumar(2, 3, 4));
24         System.out.println("Suma 2 doubles (2.5+3.5): " + suma.sumar(2.5, 3.5));

```

```
25         System.out.println("Suma 3 doubles (2.5+3.5+4.5): " + suma.sumar(2.5, 3.5, 4.5));
26     }
27 }
```

1.2.2. Diagrama de clases

Suma
+ sumar(a: int, b: int): int + sumar(a: int, b: int, c: int): int + sumar(a: double, b: double): double + sumar(a: double, b: double, c: double): double + main(args: String[])\$

1.2.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_2_10

2. Ejercicio 2.11. Sobrecarga de constructores

2.1. Enunciado (Ejemplo del Texto): Clase ArtículoCientífico

El ejemplo muestra la sobrecarga de constructores, utilizando `this()` para llamar a un constructor con menos parámetros desde uno con mayor cantidad de datos.

2.1.1. Código Fuente

```
1  /**
2   * Esta clase define objetos de tipo ArtículoCientífico con un título,
3   * autor, tres palabras clave, año de publicación y un resumen.
4   * @version 1.2/2020
5   */
6  public class ArtículoCientífico {
7      String título; // Atributo que define el título de un artículo científico
8      String autor; // Atributo que define el autor de un artículo científico
9      // Atributo que define las palabras clave de un artículo científico
10     String[] palabrasClaves = new String[3];
11     String publicación; /* Atributo que define la publicación que incluye
12     el artículo científico */
13     int año; /* Atributo que define el año de publicación de un artículo
14     científico */
15     String resumen; /* Atributo que define el resumen de un artículo
16     científico */
17
18     /**
19      * Constructor de la clase ArtículoCientífico
20      * @param título Parámetro que define el título del artículo científico
21      * @param autor Parámetro que define el autor del artículo científico
22      */
23     public ArtículoCientífico(String título, String autor) {
24         this.título = título;
25         this.autor = autor;
26     }
27
28     /**
29      * Constructor sobrecargado de la clase ArtículoCientífico
30      * @param título Parámetro que define el título del artículo científico
31      * @param autor Parámetro que define el autor del artículo científico
32      * @param palabrasClaves Parámetro que define un listado de
33      * palabras clave para el artículo científico
34      * @param publicación Parámetro que define el nombre de la
35      * publicación a la que pertenece el artículo científico
36      * @param año Parámetro que define el año de publicación del
37      * artículo científico
38      */
39     public ArtículoCientífico(String título, String autor, String[] palabrasClaves, String publicación,
40         ↪ int año) {
41         this(título, autor); // Invoca al constructor sobrecargado
42         this.palabrasClaves = palabrasClaves;
43         this.publicación = publicación;
44         this.año = año;
45     }
46
47     /**
48      * Constructor sobrecargado de la clase ArtículoCientífico
49      * @param título Parámetro que define el título del artículo científico
50      * @param autor Parámetro que define el autor del artículo científico
51      * @param palabrasClaves Parámetro que define un listado de
52      * palabras clave para el artículo científico
53      * @param publicación Parámetro que define el nombre de la
54      * publicación a la que pertenece el artículo científico
55      * @param año Parámetro que define el año de publicación del
56      * artículo científico
57      * @param resumen Parámetro que define el resumen del artículo
58      * científico
59      */
60     public ArtículoCientífico(String título, String autor, String[] palabrasClaves, String publicación,
61         ↪ int año, String resumen) {
62         this(título, autor, palabrasClaves, publicación, año); /* Invoca al
63         constructor sobrecargado */
64         this.resumen = resumen;
65     }
66
67     /**
68      * Método que imprime en pantalla los datos de un artículo científico
69      */
70     public void imprimir() {
71         System.out.println("Título del artículo = " + título);
72         System.out.println("Autor del artículo = " + autor);
73         System.out.println("Palabras clave = ");
74         // Recorre el array para imprimir cada palabra clave
75         for (int i = 0; i < palabrasClaves.length; i++) {
76             System.out.println(palabrasClaves[i]);
77         }
78         System.out.println("Publicación = " + publicación);
79         System.out.println("Año = " + año);
80     }
81 }
```

```

78         System.out.println("Resumen = " + resumen);
79     }
80
81     /**
82      * Método main que instancia un artículo científico y muestra sus
83      * datos en pantalla
84      */
85     public static void main (String args[]) {
86         String[] palabras = {"Física", "Espacio", "Tiempo"};
87         ArtículoCientífico artículo = new ArtículoCientífico("La teoría especial de la relatividad", "
            ↳ Albert Einstein", palabras, "Anales de Física", 1913, "Las leyes de la física son las
            ↳ mismas en todos los sistemas de referencia inerciales.");
88         artículo.imprimir();
89     }
90 }

```

2.1.2. Diagrama de clases

ArtículoCientífico
- título: String - autor: String - palabrasClaves: String[] - publicación: String - año: int - resumen: String
+ ArtículoCientífico(título: String, autor: String) + ArtículoCientífico(título: String, autor: String, palabrasClaves: String[], publicación: String, año: int) + ArtículoCientífico(título: String, autor: String, palabrasClaves: String[], publicación: String, año: int, resumen: String) + imprimir(): void + main(args: String[])\$

2.1.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_2_11/ArticuloCientifico.java

2.2. Ejercicio Propuesto: Clases Empleado y Caja

Definición de clases con constructores sobrecargados utilizando Empleado y Caja.

2.2.1. Código Fuente

Clase Empleado:

```

1  package Ejercicio_2_11;
2
3  public class Empleado {
4      private int identificador;
5      private String nombre;
6      private String apellidos;
7      private int edad;
8
9      public Empleado() {
10         this.identificador = 100;
11         this.nombre = "Nuevo empleado";
12         this.apellidos = "Nuevo empleado";
13         this.edad = 18;
14     }
15
16     public Empleado(int identificador, String nombre, String apellidos, int edad) {
17         this.identificador = identificador;
18         this.nombre = nombre;
19         this.apellidos = apellidos;
20         this.edad = edad;
21     }
22
23     public void imprimir() {
24         System.out.println("ID: " + identificador + ", Nombre: " + nombre + ", Apellidos: " + apellidos +
            ↳ ", Edad: " + edad);
25     }
26
27     public static void main(String[] args) {
28         Empleado emp1 = new Empleado();
29         Empleado emp2 = new Empleado(101, "Juan", "Perez", 30);
30         emp1.imprimir();
31         emp2.imprimir();
32     }
33 }

```

Clase Caja:

```

1 package Ejercicio_2_11;
2
3 public class Caja {
4     private double base;
5     private double anchura;
6     private double altura;
7     private String tipo;
8
9     public Caja(double base, double anchura, double altura) {
10         this.base = base;
11         this.anchura = anchura;
12         this.altura = altura;
13         this.tipo = "Desconocido";
14     }
15
16     public Caja() {
17         this.base = 0;
18         this.anchura = 0;
19         this.altura = 0;
20         this.tipo = "Desconocido";
21     }
22
23     public Caja(double longitud) {
24         this.base = longitud;
25         this.anchura = longitud;
26         this.altura = longitud;
27         this.tipo = "Desconocido";
28     }
29
30     public Caja(double base, double anchura, double altura, String tipo) {
31         this(base, anchura, altura);
32         this.tipo = tipo;
33     }
34
35     public void imprimir() {
36         System.out.println("Base: " + base + ", Anchura: " + anchura + ", Altura: " + altura + ", Tipo: "
37             ↪ + tipo);
38     }
39
40     public static void main(String[] args) {
41         Caja caja1 = new Caja(10.5, 20.0, 15.5);
42         Caja caja2 = new Caja();
43         Caja caja3 = new Caja(12.0);
44         Caja caja4 = new Caja(10.0, 10.0, 10.0, "Cubo");
45
46         caja1.imprimir();
47         caja2.imprimir();
48         caja3.imprimir();
49         caja4.imprimir();
50     }
51 }

```

2.2.2. Diagrama de clases

Empleado
- identificador: int - nombre: String - apellidos: String - edad: int
+ Empleado() + Empleado(identificador: int, nombre: String, apellidos: String, edad: int) + imprimir(): void + main(args: String[])\$

Caja
- base: double - anchura: double - altura: double - tipo: String
+ Caja(base: double, anchura: double, altura: double) + Caja() + Caja(longitud: double) + Caja(base: double, anchura: double, altura: double, tipo: String) + imprimir(): void + main(args: String[])\$

2.2.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_2_11

3. Ejercicio 4.4. Polimorfismo

3.1. Enunciado (Ejemplo del Texto): Clase Profesor y ProfesorTitular

Instanciación de la clase ProfesorTitular asignada a una referencia de tipo Profesor. Se ejecutan métodos de polimorfismo dinámico.

3.1.1. Código Fuente

Clase Profesor:

```
1 package Profesores;
2
3 public class Profesor {
4     protected void imprimir() {
5         System.out.println("Es un profesor.");
6     }
7 }
```

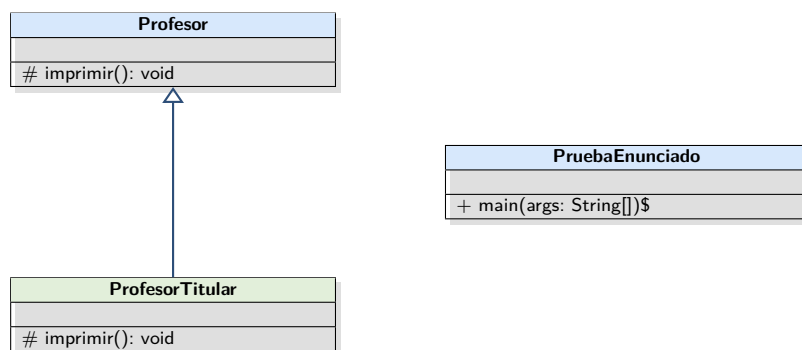
Clase ProfesorTitular:

```
1 package Profesores;
2
3 public class ProfesorTitular extends Profesor {
4     protected void imprimir() {
5         System.out.println("Es un profesor titular.");
6     }
7 }
```

Clase PruebaEnunciado:

```
1 package Profesores;
2
3 /**
4  * Esta clase prueba las clase Profesor y ProfesorTitular utilizando el
5  * polimorfismo (Ejemplo del enunciado 4.4)
6  * @version 1.2/2020
7  */
8 public class PruebaEnunciado {
9     /**
10      * Método main que crea un Profesor pero instanciando la clase
11      * ProfesorTitular.
12      */
13     public static void main(String[] args) {
14         Profesor profesor1 = new ProfesorTitular();
15         profesor1.imprimir();
16     }
17 }
```

3.1.2. Diagrama de clases



3.1.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_4/Profesores

3.2. Ejercicio Propuesto: Casting en Prueba

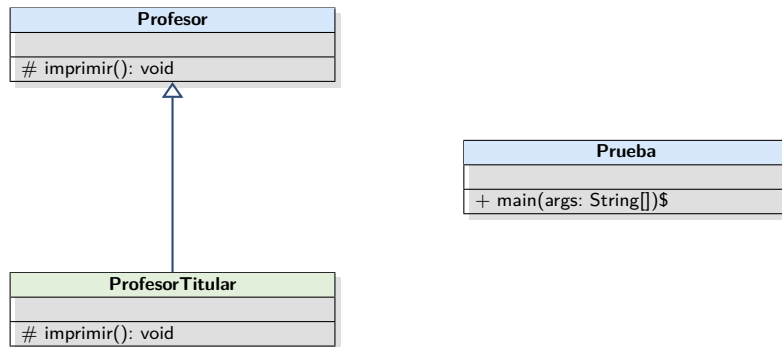
Uso del casteo manual explícito para cambiar el tipo estático de la referencia de regreso a la clase original.

3.2.1. Código Fuente

Clase Prueba (Propuesta):

```
1 package Profesores;
2
3 public class Prueba {
4     public static void main(String[] args) {
5         Profesor profesor1 = new ProfesorTitular();
6         Profesor profesor2 = (Profesor) profesor1;
7         profesor2.imprimir();
8
9         // El resultado de la ejecución es "Es un profesor titular." debido al polimorfismo,
10        // ya que el objeto subyacente instanciado en memoria es un ProfesorTitular.
11        // A pesar del casting, en tiempo de ejecución se invoca el método sobreescrito de la subclase.
12    }
13 }
```

3.2.2. Diagrama de clases



3.2.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_4

4. Ejercicio 4.6. Métodos polimórficos

4.1. Enunciado (Ejemplo del Texto): Error de tipo estático

El ejemplo muestra que invocar un método de una clase hija que no existe en la clase padre genera un error de compilación si la variable pertenece a la clase padre. **Nota de implementación:** Se ha agregado un casteo (ProfesorTitular) en el código fuente para corregir el error intencional del libro.

4.1.1. Código Fuente

Clase Profesor:

```
1 package Profesores;
2
3 public class Profesor {
4     protected void imprimir() {
5         System.out.println("Es un profesor.");
6     }
7 }
```

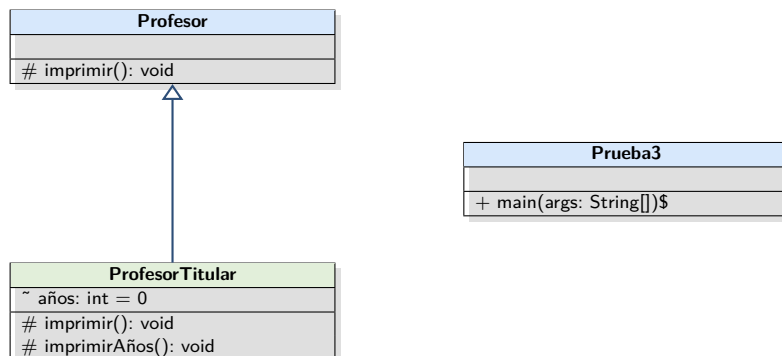
Clase ProfesorTitular:

```
1 package Profesores;
2
3 public class ProfesorTitular extends Profesor {
4     int años = 0;
5
6     protected void imprimir() {
7         System.out.println("Es un profesor titular.");
8     }
9
10    protected void imprimirAños() {
11        System.out.println("Años = " + años);
12    }
13 }
```

Clase Prueba3:

```
1 package Profesores;
2
3 /**
4  * Esta clase prueba las clases Profesor y ProfesorTitular utilizando
5  * métodos polimórficos
6  * @version 1.2/2020
7  */
8 public class Prueba3 {
9     /**
10     * Método main que crea un ProfesorTitular pero en un objeto tipo
11     * Profesor.
12     * En el texto original este código no compilaba intencionalmente,
13     * pero se ha aplicado un casteo para corregirlo como solución.
14     */
15     public static void main(String[] args) {
16         Profesor profesor1 = new ProfesorTitular();
17         ((ProfesorTitular) profesor1).imprimirAños();
18     }
19 }
```

4.1.2. Diagrama de clases



4.1.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_6/Profesores

4.2. Ejercicio Propuesto: Vector Genérico

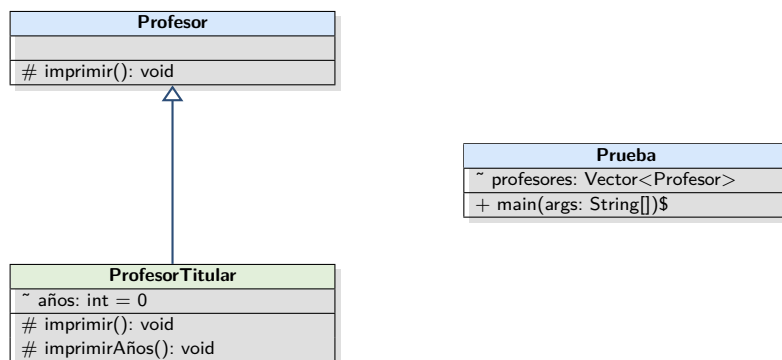
Uso de un Vector sin genéricos para almacenar objetos Profesor y usar polimorfismo dinámico al iterarlo.

4.2.1. Código Fuente

Clase Prueba (Propuesta):

```
1 package Profesores;
2 import java.util.*;
3
4 public class Prueba {
5     Vector<Profesor> profesores;
6
7     public static void main(String[] args) {
8         Prueba prueba = new Prueba();
9         prueba.profesores = new Vector<Profesor>();
10        Profesor profesor1 = new Profesor();
11        ProfesorTitular profesor2 = new ProfesorTitular();
12
13        prueba.profesores.add(profesor1);
14        prueba.profesores.add(profesor2);
15
16        for(int i = 0; i < prueba.profesores.size(); i++) {
17            Profesor p = (Profesor) prueba.profesores.elementAt(i);
18            p.imprimir();
19        }
20
21        // Resultado esperado:
22        // Es un profesor.
23        // Es un profesor titular.
24
25        // Explicación:
26        // El bucle recorre el vector que contiene elementos del tipo base 'Profesor'.
27        // En la primera iteración, 'p' hace referencia a un objeto 'Profesor', llamando a su método
28        //   ↳ original.
29        // En la segunda iteración, 'p' hace referencia a un objeto 'ProfesorTitular'. Debido al
30        //   ↳ polimorfismo,
31        // al invocar 'imprimir()' se ejecuta la versión redefinida en la subclase, a pesar de que la
32        //   ↳ referencia
33        // sea de tipo 'Profesor'.
34    }
35 }
```

4.2.2. Diagrama de clases



4.2.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_6

5. Ejercicio 4.7. Clases abstractas

5.1. Enunciado (Ejemplo del Texto): Jerarquía Animales

El ejemplo muestra cómo crear una jerarquía usando clases abstractas como Animal, Cánido y Felino para categorizar clases concretas.

5.1.1. Código Fuente

Clase Animal:

```
1 package Animales;
2
3 /**
4  * Esta clase abstracta denominada Animal modela un animal genérico
5  * que cuenta con atributos como un sonido, alimentos que consume,
6  * un hábitat y un nombre científico.
7  * @version 1.2/2020
8  */
9 public abstract class Animal {
10     protected String sonido; /* Atributo que identifica el sonido emitido por un animal */
11     protected String alimentos; /* Atributo que identifica los alimentos que consume un animal */
12     protected String hábitat; /* Atributo que identifica el hábitat de un animal */
13     protected String nombreCientífico; /* Atributo que identifica el nombre científico de un animal */
14
15     /**
16      * Método abstracto que permite obtener el nombre científico del animal
17      * @return El nombre científico del animal
18      */
19     public abstract String getNombreCientífico();
20
21     /**
22      * Método abstracto que permite obtener el sonido producido por el animal
23      * @return El sonido producido por el animal
24      */
25     public abstract String getSonido();
26
27     /**
28      * Método abstracto que permite obtener los alimentos que consume un animal
29      * @return Los alimentos que consume el animal
30      */
31     public abstract String getAlimentos();
32
33     /**
34      * Método abstracto que permite obtener el hábitat de un animal
35      * @return El hábitat del animal
36      */
37     public abstract String getHábitat();
38 }
```

Clases Cánido y Felino:

```
1 package Animales;
2
3 /**
4  * Esta clase abstracta denominada Cánido modela esta familia de
5  * animales. Es una subclase de Animal.
6  * @version 1.2/2020
7  */
8 public abstract class Canido extends Animal {
9 }
```

```
1 package Animales;
2
3 /**
4  * Esta clase abstracta denominada Felino modela esta familia de
5  * animales. Es una subclase de Animal.
6  * @version 1.2/2020
7  */
8 public abstract class Felino extends Animal {
9 }
```

Clases Concretas (Perro, Lobo, León, Gato):

```
1 package Animales;
2
3 /**
4  * Esta clase concreta denominada Perro es una subclase de Cánido.
5  * @version 1.2/2020
6  */
7 public class Perro extends Canido {
8     /**
9      * Método que devuelve un String con el sonido de un perro
10     * @return Un valor String con el sonido de un perro: "Ladrado"
11     */
12     public String getSonido() {
```

```

13         return "Ladrado";
14     }
15
16     /**
17      * Método que devuelve un String con los alimentos de un perro
18      * @return Un valor String con la alimentación de un perro: "Carnívoro"
19      */
20     public String getAlimentos() {
21         return "Carnívoro";
22     }
23
24     /**
25      * Método que devuelve un String con el hábitat de un perro
26      * @return Un valor String con el hábitat de un perro: "Doméstico"
27      */
28     public String getHábitat() {
29         return "Doméstico";
30     }
31
32     /**
33      * Método que devuelve un String con el nombre científico de un perro
34      * @return Un valor String con el nombre científico de un perro:
35      * "Canis lupus familiaris"
36      */
37     public String getNombreCientífico() {
38         return "Canis lupus familiaris";
39     }
40 }

```

```

1 package Animales;
2
3 /**
4  * Esta clase concreta denominada Lobo es una subclase de Cánido.
5  * @version 1.2/2020
6  */
7 public class Lobo extends Canido {
8     /**
9      * Método que devuelve un String con el sonido de un lobo
10     * @return Un valor String con el sonido de un lobo: "Aullido"
11     */
12     public String getSonido() {
13         return "Aullido";
14     }
15
16     /**
17      * Método que devuelve un String con los alimentos de un lobo
18      * @return Un valor String con el tipo de alimentación de un lobo:
19      * "Carnívoro"
20      */
21     public String getAlimentos() {
22         return "Carnívoro";
23     }
24
25     /**
26      * Método que devuelve un String con el hábitat de un lobo
27      * @return Un valor String con el hábitat de un lobo: "Bosque"
28      */
29     public String getHábitat() {
30         return "Bosque";
31     }
32
33     /**
34      * Método que devuelve un String con el nombre científico de un lobo
35      * @return Un valor String con el nombre científico de un lobo:
36      * "Canis lupus"
37      */
38     public String getNombreCientífico() {
39         return "Canis lupus";
40     }
41 }

```

```

1 package Animales;
2
3 /**
4  * Esta clase concreta denominada León es una subclase de Felino.
5  * @version 1.2/2020
6  */
7 public class Leon extends Felino {
8     /**
9      * Método que devuelve un String con el sonido de un león
10     * @return Un valor String con el sonido de un león: "Rugido"
11     */
12     public String getSonido() {
13         return "Rugido";
14     }
15
16     /**
17      * Método que devuelve un String con los alimentos de un león
18      * @return Un valor String con la alimentación de un león: "Carnívoro"
19      */

```

```

20     public String getAlimentos() {
21         return "Carnívoro";
22     }
23
24     /**
25      * Método que devuelve un String con el hábitat de un león
26      * @return Un valor String con el hábitat de un león: "Praderas"
27      */
28     public String getHábitat() {
29         return "Praderas";
30     }
31
32     /**
33      * Método que devuelve un String con el nombre científico de un león
34      * @return Un valor String con el nombre científico de un león:
35      * "Panthera leo"
36      */
37     public String getNombreCientífico() {
38         return "Panthera leo";
39     }
40 }

```

```

1 package Animales;
2
3 /**
4  * Esta clase concreta denominada Gato es una subclase de Felino.
5  * @version 1.2/2020
6  */
7 public class Gato extends Felino {
8     /**
9      * Método que devuelve un String con el sonido de un gato
10     * @return Un valor String con el sonido de un gato: "Mauullido"
11     */
12     public String getSonido() {
13         return "Mauullido";
14     }
15
16     /**
17      * Método que devuelve un String con los alimentos de un gato
18      * @return Un valor String con la alimentación de un gato: "Ratones"
19      */
20     public String getAlimentos() {
21         return "Ratones";
22     }
23
24     /**
25      * Método que devuelve un String con el hábitat de un gato
26      * @return Un valor String con el hábitat de un gato: "Doméstico"
27      */
28     public String getHábitat() {
29         return "Doméstico";
30     }
31
32     /**
33      * Método que devuelve un String con el nombre científico de un gato
34      * @return Un valor String con el nombre científico de un gato:
35      * "Felis silvestris catus"
36      */
37     public String getNombreCientífico() {
38         return "Felis silvestris catus";
39     }
40 }

```

Clase Prueba:

```

1 package Animales;
2
3 /**
4  * Esta clase prueba diferentes animales y sus métodos.
5  * @version 1.2/2020
6  */
7 public class Prueba {
8     /**
9      * Método main que crea un array de varios animales y para cada uno
10     * muestra en pantalla su nombre científico, su sonido, alimentos y
11     * hábitat
12     */
13     public static void main(String[] args) {
14         Animal[] animales = new Animal[4]; /* Define un array de cuatro
15         elementos de tipo Animal */
16         animales[0] = new Gato();
17         animales[1] = new Perro();
18         animales[2] = new Lobo();
19         animales[3] = new Leon();
20
21         for (int i = 0; i < animales.length; i++) { /* Recorre el array de animales */
22             System.out.println(animales[i].getNombreCientífico());
23             System.out.println("Sonido: " + animales[i].getSonido());
24             System.out.println("Alimentos: " + animales[i].getAlimentos());
25             System.out.println("Hábitat: " + animales[i].getHábitat());
26             System.out.println();
27         }
28     }
29 }

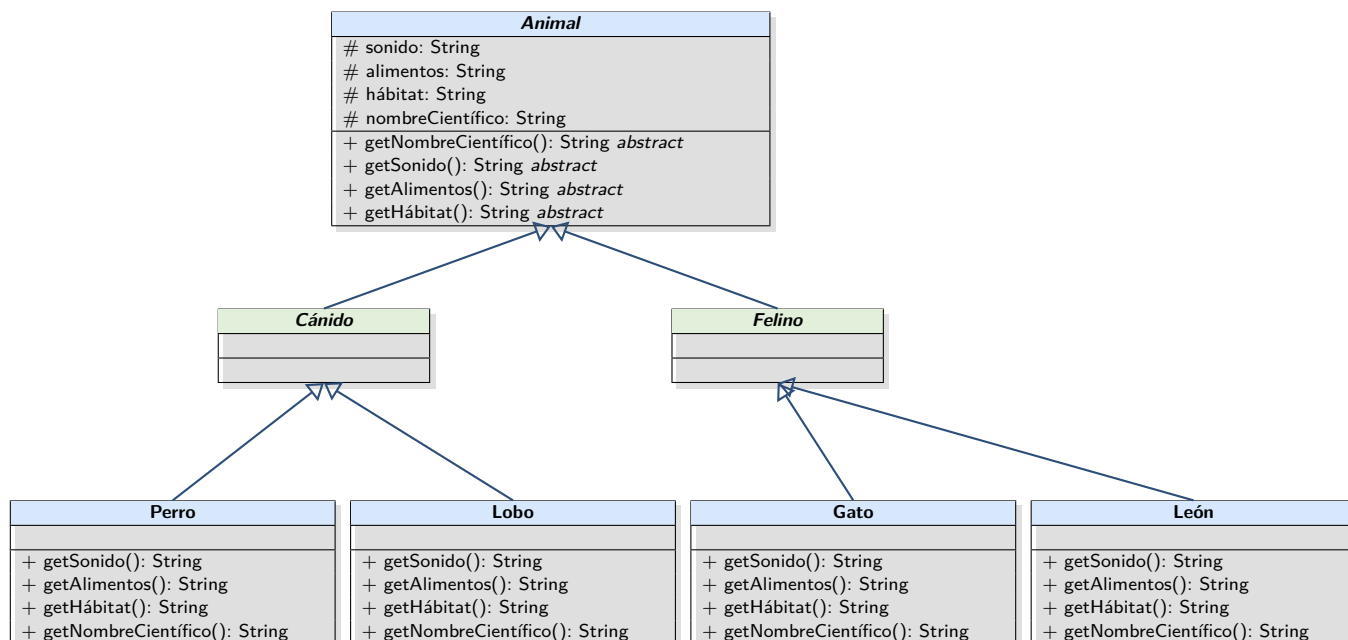
```

```

27     }
28 }
29 }

```

5.1.2. Diagrama de clases



5.1.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_7/Animales

5.2. Ejercicio Propuesto: Jerarquía Numérica

Clase abstracta Numérica y su implementación concreta en Fracción.

5.2.1. Código Fuente

Clase Numérica:

```

1 package Ejercicio_4_7;
2
3 public abstract class Numérica {
4     public abstract String toString();
5     public abstract boolean equals(Object ob);
6     public abstract Numérica sumar(Numérica número);
7     public abstract Numérica restar(Numérica número);
8     public abstract Numérica multiplicar(Numérica número);
9     public abstract Numérica dividir(Numérica número);
10 }

```

Clase Fracción:

```

1 package Ejercicio_4_7;
2
3 public class Fracción extends Numérica {
4     private int numerador;
5     private int denominador;
6
7     public Fracción(int numerador, int denominador) {
8         this.numerador = numerador;
9         this.denominador = denominador != 0 ? denominador : 1; // Prevenir división por 0
10    }
11
12    public int getNumerador() { return numerador; }
13    public int getDenominador() { return denominador; }
14
15    @Override
16    public String toString() {
17        return numerador + "/" + denominador;
18    }
19
20    @Override

```

```

21     public boolean equals(Object ob) {
22         if (ob instanceof Fracción) {
23             Fracción f = (Fracción) ob;
24             // Comparación de fracciones usando producto cruzado
25             return this.numerador * f.denominador == this.denominador * f.numerador;
26         }
27         return false;
28     }
29
30     @Override
31     public Numérica sumar(Numérica número) {
32         if (número instanceof Fracción) {
33             Fracción f = (Fracción) número;
34             int nuevoNumerador = this.numerador * f.denominador + f.numerador * this.denominador;
35             int nuevoDenominador = this.denominador * f.denominador;
36             return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
37         }
38         return null;
39     }
40
41     @Override
42     public Numérica restar(Numérica número) {
43         if (número instanceof Fracción) {
44             Fracción f = (Fracción) número;
45             int nuevoNumerador = this.numerador * f.denominador - f.numerador * this.denominador;
46             int nuevoDenominador = this.denominador * f.denominador;
47             return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
48         }
49         return null;
50     }
51
52     @Override
53     public Numérica multiplicar(Numérica número) {
54         if (número instanceof Fracción) {
55             Fracción f = (Fracción) número;
56             int nuevoNumerador = this.numerador * f.numerador;
57             int nuevoDenominador = this.denominador * f.denominador;
58             return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
59         }
60         return null;
61     }
62
63     @Override
64     public Numérica dividir(Numérica número) {
65         if (número instanceof Fracción) {
66             Fracción f = (Fracción) número;
67             int nuevoNumerador = this.numerador * f.denominador;
68             int nuevoDenominador = this.denominador * f.numerador;
69             return simplificar(new Fracción(nuevoNumerador, nuevoDenominador));
70         }
71         return null;
72     }
73
74     private Fracción simplificar(Fracción f) {
75         int mcd = mcd(Math.abs(f.getNumerador()), Math.abs(f.getDenominador()));
76         return new Fracción(f.getNumerador() / mcd, f.getDenominador() / mcd);
77     }
78
79     private int mcd(int a, int b) {
80         if (b == 0) {
81             return a;
82         }
83         return mcd(b, a % b);
84     }
85 }

```

Clase Prueba:

```

1 package Ejercicio_4_7;
2
3 public class Prueba {
4     public static void main(String[] args) {
5         Fracción f1 = new Fracción(1, 2);
6         Fracción f2 = new Fracción(3, 4);
7
8         System.out.println("Fracción 1: " + f1.toString());
9         System.out.println("Fracción 2: " + f2.toString());
10
11         System.out.println("¿Son iguales? " + f1.equals(f2));
12
13         Numérica suma = f1.sumar(f2);
14         System.out.println("Suma (1/2 + 3/4): " + suma.toString());
15
16         Numérica resta = f1.restar(f2);
17         System.out.println("Resta (1/2 - 3/4): " + resta.toString());
18
19         Numérica multiplicacion = f1.multiplicar(f2);
20         System.out.println("Multiplicación (1/2 * 3/4): " + multiplicacion.toString());
21
22         Numérica division = f1.dividir(f2);
23         System.out.println("División (1/2 / 3/4): " + division.toString());
24     }

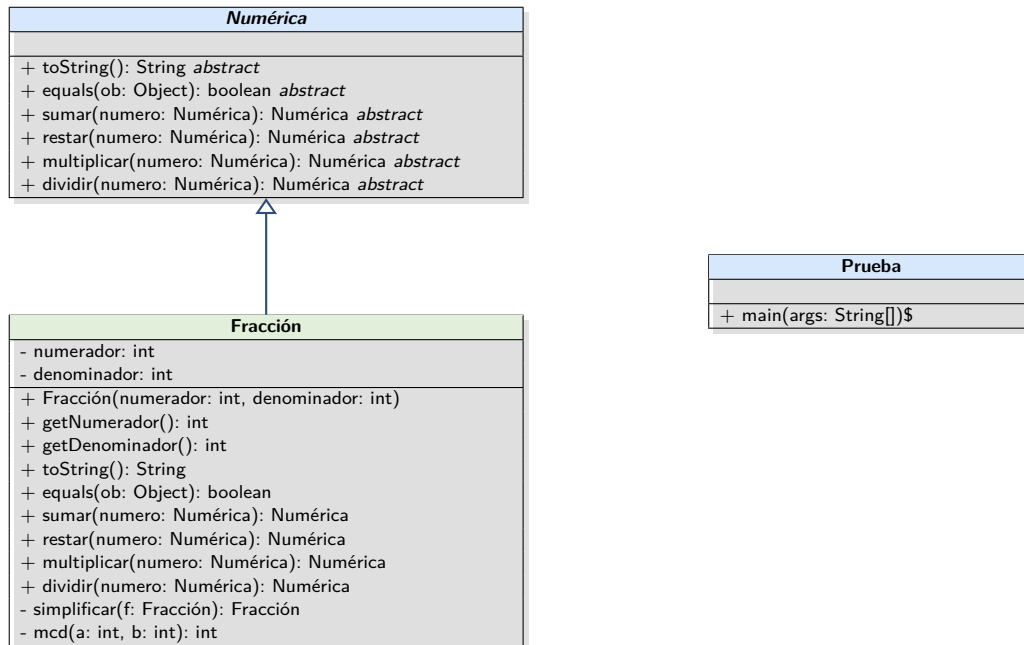
```

```

25         // Otro caso de prueba (Fracciones iguales)
26         Fracción f3 = new Fracción(2, 4);
27         System.out.println("\n'1/2 es igual a 2/4? " + f1.equals(f3));
28     }
29 }

```

5.2.2. Diagrama de clases



5.2.3. Link

https://github.com/mateolikescats/P00/tree/main/Actividad_6/Ejercicio_4_7